

Übungseinheit: Einfache Datentypen

Praktische Anwendung von Strings, Integer, Float und Boolean in Python

Dorian Zwanzig, Claude

2025-01-10

Inhaltsverzeichnis

1	Über diese Übung	3
1.1	Lernziele	3
1.2	Voraussetzungen	3
1.3	Hinweis	3
2	Vorbereitung (ca. 10 Min)	4
2.1	Schritt-für-Schritt Anleitung	4
2.2	Erfolgskontrolle	4
3	Einführung in Datentypen (ca. 15 Min)	5
3.1	Die vier grundlegenden Datentypen	5
3.2	Warum verschiedene Datentypen?	5
3.3	In dieser Übung lernen Sie:	5
4	Übungsblock A – Strings (ca. 20 Min)	6
4.1	Vorbereitung	6
4.2	Aufgabe 1: Namen speichern und ausgeben	6
4.3	Aufgabe 2: Zeichenketten verknüpfen	6
4.4	Aufgabe 3: Länge eines Strings ermitteln	7
5	Neue Funktion: len()	7
5.1	Aufgabe 4: Zeichen ersetzen	7
6	Neue Methode: .replace()	7
6.1	Aufgabe 5: f-Strings verwenden	8
7	Neues Konzept: f-Strings (formatierte Strings)	8
7.1	Aufgabe 6 (Erweitert): Mit Benutzereingabe arbeiten	9
7.2	Datei ausführen und testen	10
7.3	Musterlösungen	10
8	Übungsblock B – Integer (ca. 15 Min)	12
8.1	Vorbereitung	12
8.2	Aufgabe 1: Grundrechenarten mit ganzen Zahlen	12
9	Neue Operatoren: // und %	12
10	Neue Funktion: type()	12
10.1	Aufgabe 2 (Erweitert): Taschenrechner mit Eingabe	13
10.2	Datei ausführen und testen	14
10.3	Musterlösungen	14
11	Übungsblock C – Float (ca. 15 Min)	15
11.1	Vorbereitung	15

11.2 Aufgabe 1: Preisberechnung mit Mehrwertsteuer	15
12 Neue Funktion: round()	15
12.1 Aufgabe 2 (Erweitert): Preisrechner mit Eingabe	16
12.2 Datei ausführen und testen	16
12.3 Musterlösungen	16
13 Übungsblock D – Boolean (ca. 10 Min)	18
13.1 Vorbereitung	18
13.2 Aufgabe: Logische Verknüpfungen verstehen	18
14 Logische Operatoren: and, or, not	18
15 Vergleichsoperatoren	18
15.1 Datei ausführen und testen	19
15.2 Musterlösungen	19
16 Übungsblock E – Typprüfung & Umwandlung (ca. 15 Min)	21
16.1 Vorbereitung	21
16.2 Aufgabe: Datentypen umwandeln	21
17 Typumwandlung: str(), int(), float(), bool()	21
17.1 Datei ausführen und testen	22
17.2 Musterlösungen	22
18 Übungsblock F – Gemischte Aufgaben (ca. 25 Min)	24
18.1 Aufgabe 1: Temperaturumrechner	24
18.2 Aufgabe 2: Notenprüfung	25
18.3 Musterlösungen	25
19 Typische Anfängerfehler – Troubleshooting	27
19.1 Python-Fehler	27
19.2 VSCode-spezifische Probleme	27
19.3 Eingabefehler bei input()	27
19.4 Tipps zur Fehlervermeidung	27
20 Abschluss / Reflexion (ca. 10 Min)	28
20.1 Alle Skripte testen	28
20.2 Reflexionsfragen	28
20.3 Dateien organisieren	28
20.4 Was haben Sie gelernt?	28
20.5 Nächste Schritte	28

1 Über diese Übung

1.1 Lernziele

Nach Abschluss dieser Übung können Sie: - Variablen vom Typ String erstellen und mit Textinhalten arbeiten - Integer-Variablen für ganzzahlige Werte verwenden - Float-Variablen für Dezimalzahlen einsetzen - Boolean-Variablen für Wahrheitswerte nutzen - Den passenden Datentyp für verschiedene Anwendungsfälle auswählen - Typumwandlungen zwischen verschiedenen Datentypen durchführen - Die Funktion `type()` zur Überprüfung von Datentypen anwenden

Zeitbedarf: ca. 90 Minuten **Schwierigkeitsgrad:** * (Einsteiger)

1.2 Voraussetzungen

Für diese Übung sollten Sie folgende Konzepte sicher beherrschen: - **Grundlagen der Programmierung** (Notebook 01) - **Konsolenausgabe mit `print()`** (Notebook 02) - **Kommentare in Python** (Notebook 03) - **Variablen und Wertzuweisung** (Notebook 04)

Falls Sie unsicher sind, wiederholen Sie bitte die entsprechenden Notebooks.

1.3 Hinweis

Legen Sie mindestens für jeden Übungsblock eine eigene Datei an, idealerweise für jede Übung. Das hilft Ihnen die Übersicht zu behalten - nicht nur heute, sondern auch während der Klausurvorbereitung.

2 Vorbereitung (ca. 10 Min)

Ziel: VS Code öffnen, das Programm kennenlernen, Dateien anlegen, speichern und ausführen.

2.1 Schritt-für-Schritt Anleitung

1. **VS Code starten** (Zenworks – VSCode)
2. **Neues Projektverzeichnis erstellen:**
 - Datei – Ordner öffnen...
 - Ordner z. B. `python_grundlagen` nennen
3. **Neue Datei erstellen:**
 - Im Explorer links: Rechtsklick – Neue Datei
 - Dateiname: `hello_world.py`
4. **Code eingeben:**

```
print("Hello World!")
```

5. **Datei speichern:**
 - Windows/Linux: `Strg + S`
 - macOS: `Cmd + S`
6. **Terminal öffnen:**
 - Windows: `Strg + ö`
 - Alternativ: *Terminal – Neues Terminal*
7. **Datei ausführen:**

```
python hello_world.py          # Windows
python3 hello_world.py         # macOS/Linux
```

8. **Erwartete Ausgabe überprüfen:**

Hello World!

9. **Terminal-Ausgabe löschen:**

```
cls          # Windows
clear        # macOS/Linux
```

2.2 Erfolgskontrolle

- ☒ VS Code ist geöffnet
- ☒ Projektordner wurde erstellt
- ☒ Erste Python-Datei läuft
- ☒ "Hello World!" wird ausgegeben

3 Einführung in Datentypen (ca. 15 Min)

3.1 Die vier grundlegenden Datentypen

Python kennt verschiedene Arten von Daten. Die wichtigsten für den Anfang sind:

Tabelle 1: Die vier grundlegenden Datentypen in Python
{#tbl:datentypen}

Datentyp	Beschreibung	Beispiel
String	Text, in Anführungszeichen	"Hallo", 'Welt'
Integer	Ganze Zahl	42, -17, 0
Float	Kommazahl (mit Punkt!)	3.14, -2.5, 0.0
Boolean	Wahrheitswert	True, False

3.2 Warum verschiedene Datentypen?

- **Effizienz:** Zahlen brauchen weniger Speicher als Text
- **Operationen:** Mit Zahlen kann man rechnen, mit Text nicht
- **Klarheit:** Der Code wird verständlicher
- **Fehlerprävention:** Python warnt bei falscher Verwendung

3.3 In dieser Übung lernen Sie:

- Variablen mit verschiedenen Datentypen erstellen
- Mit den Datentypen arbeiten
- Zwischen Typen umwandeln
- Typische Fehler vermeiden

4 Übungsblock A – Strings (ca. 20 Min)

4.1 Vorbereitung

1. Neue Datei erstellen: `strings_uebung.py`
 2. Datei im Editor öffnen
-

4.2 Aufgabe 1: Namen speichern und ausgeben

4.2.1 Aufgabenstellung

Erstellen Sie eine Variable mit dem Namen `name` und weisen Sie ihr Ihren Vornamen als Text zu. Geben Sie den Inhalt der Variable auf dem Bildschirm aus.

4.2.2 Hilfestellungen

Problemanalyse

- Was soll gespeichert werden? – Ein Textinhalt (Ihr Vorname)
- Welcher Datentyp wird benötigt? – String (Text in Anführungszeichen)
- Wie gibt man etwas aus? – Mit der `print()`-Funktion

Lösungsschritte

1. Eine Variable mit dem Namen `name` erstellen
2. Der Variable einen String-Wert zuweisen (Ihren Vornamen in Anführungszeichen)
3. Die Variable mit `print()` ausgeben

Pseudocode

```
ERSTELLE Variable name MIT Wert "IhrVorname"  
AUSGABE name
```

4.2.3 Erwartete Ausgabe

Lina

4.3 Aufgabe 2: Zeichenketten verknüpfen

4.3.1 Aufgabenstellung

Erstellen Sie zwei Variablen `vorname` und `nachname` mit entsprechenden Texten. Verknüpfen Sie beide zu einem vollständigen Namen (mit Leerzeichen dazwischen) und speichern Sie das Ergebnis in einer neuen Variable `voller_name`. Geben Sie das Ergebnis aus.

4.3.2 Hilfestellungen

Problemanalyse

- Was soll erreicht werden? – Zwei separate Texte zu einem kombinieren
- Welche Daten werden benötigt? – Vorname und Nachname als separate Variablen
- Was fehlt zwischen den Namen? – Ein Leerzeichen als Trennung

Lösungsschritte

1. Variable `vorname` mit Ihrem Vornamen erstellen
2. Variable `nachname` mit Ihrem Nachnamen erstellen
3. Beide Strings mit `+` verknüpfen, Leerzeichen " " dazwischen einfügen
4. Das Ergebnis in einer neuen Variable `voller_name` speichern
5. Das Ergebnis ausgeben

Pseudocode

```
ERSTELLE Variable vorname MIT Wert "Lina"
ERSTELLE Variable nachname MIT Wert "Schmidt"
ERSTELLE Variable voller_name MIT vorname + " " + nachname
AUSGABE voller_name
```

4.3.3 Erwartete Ausgabe

Lina Schmidt

4.4 Aufgabe 3: Länge eines Strings ermitteln

5 Neue Funktion: len()

Die Funktion `len()` (von “length” = Länge) gibt die Anzahl der Zeichen in einem String zurück.

Syntax: `len(text)`

Beispiel:

```
name = "Anna"
laenge = len(name)
print(laenge) # Ausgabe: 4
```

Wichtig: Leerzeichen, Zahlen und Sonderzeichen werden mitgezählt!

5.0.1 Aufgabenstellung

Finden Sie heraus, wie viele Zeichen der vollständige Name (inklusive Leerzeichen) enthält und geben Sie diese Zahl aus.

5.0.2 Hilfestellungen

Problemanalyse

- Was soll ermittelt werden? – Die Anzahl der Zeichen in einem String
- Welche Funktion hilft dabei? – Die Funktion `len()` zählt Zeichen
- Was wird mitgezählt? – Alle Zeichen inklusive Leerzeichen

Lösungsschritte

1. Die Variable `voller_name` aus der vorherigen Aufgabe verwenden
2. Die Funktion `len()` auf die Variable anwenden
3. Das Ergebnis mit `print()` ausgeben

Pseudocode

```
anzahl_zeichen = LÄNGE VON voller_name
AUSGABE anzahl_zeichen
```

5.0.3 Erwartete Ausgabe

12

5.1 Aufgabe 4: Zeichen ersetzen

6 Neue Methode: .replace()

Die Methode `.replace()` ersetzt Textstellen in einem String.

Syntax: `text.replace("alt", "neu")`

Beispiel:

```
satz = "Ich mag Äpfel"
neuer_satz = satz.replace("Äpfel", "Birnen")
print(neuer_satz)  # Ausgabe: Ich mag Birnen
print(satz)        # Ausgabe: Ich mag Äpfel (Original unverändert!)
```

Wichtig: Der ursprüngliche String bleibt unverändert. Das Ergebnis wird zurückgegeben.

6.0.1 Aufgabenstellung

Ersetzen Sie im vollständigen Namen "Lina" durch "Mara" und geben Sie das Ergebnis aus.

6.0.2 Hilfestellungen

Problemanalyse

- Was soll passieren? – Ein Textbestandteil soll durch einen anderen ersetzt werden
- Welche Methode hilft dabei? – Die Methode `.replace()` ersetzt Textstellen
- Was passiert mit dem Original? – Es bleibt unverändert, das Ergebnis muss ausgegeben werden

Lösungsschritte

1. Die Variable `voller_name` aus der vorherigen Aufgabe verwenden
2. Die Methode `.replace()` aufrufen mit "Lina" als altem und "Mara" als neuem Wert
3. Das Ergebnis direkt ausgeben oder in einer neuen Variable speichern

Pseudocode

```
neuer_name = voller_name MIT ERSETZUNG "Lina" DURCH "Mara"
AUSGABE neuer_name
```

6.0.3 Erwartete Ausgabe

Mara Schmidt

6.1 Aufgabe 5: f-Strings verwenden

7 Neues Konzept: f-Strings (formatierte Strings)

f-Strings ermöglichen es, Variablen direkt in Texte einzufügen.

Syntax: `f"Text {variable} mehr Text"`

Beispiel:

```
name = "Max"
alter = 25
print(f"Hallo {name}, du bist {alter} Jahre alt.")
# Ausgabe: Hallo Max, du bist 25 Jahre alt.
```

Mit Berechnungen:

```
preis = 10
print(f"Der doppelte Preis ist {preis * 2} Euro.")
# Ausgabe: Der doppelte Preis ist 20 Euro.
```

Wichtig: Das `f` vor dem String nicht vergessen!

7.0.1 Aufgabenstellung

Geben Sie einen Satz aus, der den Vornamen und die Anzahl seiner Buchstaben enthält. Nutzen Sie dafür f-Strings.

Beispiel: "Hallo Lina, dein Name hat 4 Buchstaben."

7.0.2 Hilfestellungen

Problemanalyse

- Was soll ausgegeben werden? – Ein Satz mit eingebetteten Variablen und Berechnungen
- Wie fügt man Variablen in Text ein? – Mit f-Strings: `f"Text {variable}"`
- Wie berechnet man die Buchstabenanzahl? – Mit `len(vorname)` direkt im f-String

Lösungsschritte

1. Einen f-String erstellen (beginnt mit `f` vor den Anführungszeichen)
2. Den Text mit Platzhaltern `{}` für Variable und Berechnung schreiben
3. In die erste Klammer `{vorname}` einfügen
4. In die zweite Klammer `{len(vorname)}` einfügen
5. Mit `print()` ausgeben

Pseudocode

```
AUSGABE f"Hallo {vorname}, dein Name hat {LÄNGE VON vorname} Buchstaben."
```

7.0.3 Erwartete Ausgabe

Hallo Lina, dein Name hat 4 Buchstaben.

7.1 Aufgabe 6 (Erweitert): Mit Benutzereingabe arbeiten

Schwierigkeit: Mittel

7.1.1 Aufgabenstellung

Ändern Sie das Programm so, dass der Benutzer seinen Vor- und Nachnamen selbst eingeben kann. Das Programm soll dann: - Eine persönliche Begrüßung ausgeben - Die Gesamtzahl der Zeichen (inklusive Leerzeichen) anzeigen

7.1.2 Hilfestellungen

Problemanalyse

- Woher kommen die Daten? – Vom Benutzer über die Konsole
- Welche Funktion liest Eingaben? – `input("Frage")`
- Was wird dann mit den Eingaben gemacht? – Verknüpfen, Länge berechnen, ausgeben

Lösungsschritte

1. Trennlinie ausgeben mit `print("\n--- Jetzt mit Eingabe ---")`
2. Benutzer nach Vornamen fragen und in Variable speichern
3. Benutzer nach Nachnamen fragen und in Variable speichern
4. Vor- und Nachname zu `voller_name` verknüpfen
5. Persönliche Begrüßung mit f-String ausgeben
6. Zeichenanzahl mit `len()` berechnen und ausgeben

Pseudocode

```
AUSGABE "--- Jetzt mit Eingabe ---"
vorname = EINGABE "Wie ist dein Vorname? "
nachname = EINGABE "Wie ist dein Nachname? "
voller_name = vorname + " " + nachname
AUSGABE f"Hallo {voller_name}!"
AUSGABE f"Dein vollständiger Name hat {LÄNGE VON voller_name} Zeichen..."
```

7.1.3 Beispiel-Ausgabe (bei Eingabe "Anna" und "Müller")

```
--- Jetzt mit Eingabe ---  
Wie ist dein Vorname? Anna  
Wie ist dein Nachname? Müller  
Hallo Anna Müller!  
Dein vollständiger Name hat 11 Zeichen (inkl. Leerzeichen).
```

7.2 Datei ausführen und testen

```
python strings_uebung.py
```

Gesamte erwartete Ausgabe der Aufgaben 1-5:

```
Lina  
Lina Schmidt  
12  
Mara Schmidt  
Hallo Lina, dein Name hat 4 Buchstaben.
```

7.3 Musterlösungen

Lösung zu Aufgabe 1

```
# 1. Einen Namen speichern und ausgeben  
name = "Lina"  
print(name)
```

Erklärung: - Variable `name` wird mit dem String "Lina" initialisiert - `print()` gibt den Inhalt der Variable aus

Lösung zu Aufgabe 2

```
# 2. Zeichenketten verknüpfen  
vorname = "Lina"  
nachname = "Schmidt"  
voller_name = vorname + " " + nachname  
print(voller_name)
```

Erklärung: - Drei Strings werden mit `+` verknüpft - Das Leerzeichen `" "` trennt Vor- und Nachname

Lösung zu Aufgabe 3

```
# 3. Länge eines Strings prüfen  
print(len(voller_name))
```

Erklärung: - `len()` zählt alle Zeichen inklusive Leerzeichen

Lösung zu Aufgabe 4

```
# 4. Zeichen ersetzen  
print(voller_name.replace("Lina", "Mara"))
```

Erklärung: - `.replace()` erstellt einen neuen String mit der Ersetzung - Der ursprüngliche String bleibt unverändert

Lösung zu Aufgabe 5

```
# 5. f-Strings verwenden  
print(f"Hallo {vorname}, dein Name hat {len(vorname)} Buchstaben.")
```

Erklärung: - `f` vor dem String aktiviert die Formatierung - In `{}` können Variablen und Ausdrücke stehen

Lösung zu Aufgabe 6 (Erweitert)

6. Mit Benutzereingabe arbeiten

```
print("\n--- Jetzt mit Eingabe ---")
vorname = input("Wie ist dein Vorname? ")
nachname = input("Wie ist dein Nachname? ")
voller_name = vorname + " " + nachname
print(f"Hallo {voller_name}!")
print(f"Dein vollständiger Name hat {len(voller_name)} Zeichen (inkl. Leerzeichen).")
```

Erklärung: - `input()` wartet auf Benutzereingabe - Die Eingabe wird als String zurückgegeben - f-Strings kombinieren Text mit Variablen

8 Übungsblock B – Integer (ca. 15 Min)

8.1 Vorbereitung

1. Neue Datei erstellen: `integer_uebung.py`
 2. Datei im Editor öffnen
-

8.2 Aufgabe 1: Grundrechenarten mit ganzen Zahlen

9 Neue Operatoren: `//` und `%`

Ganzzahlige Division (`//`): Teilt zwei Zahlen und gibt nur den ganzen Teil zurück.

```
print(10 // 3)  # Ausgabe: 3 (nicht 3.333...)
print(17 // 5)  # Ausgabe: 3
```

Modulo (`%`): Gibt den Rest einer Division zurück.

```
print(10 % 3)   # Ausgabe: 1 (10 geteilt durch 3 = 3 Rest 1)
print(17 % 5)   # Ausgabe: 2 (17 geteilt durch 5 = 3 Rest 2)
```

Anwendung: Mit `%` kann man z.B. prüfen, ob eine Zahl gerade ist: `zahl % 2 == 0`

10 Neue Funktion: `type()`

Die Funktion `type()` zeigt den Datentyp einer Variable an.

Syntax: `type(variable)`

Beispiel:

```
x = 42
print(type(x))      # Ausgabe: <class 'int'>
y = "Hallo"
print(type(y))      # Ausgabe: <class 'str'>
```

10.0.1 Aufgabenstellung

Erstellen Sie zwei Variablen `a` und `b` mit den Werten 10 und 4. Führen Sie folgende Berechnungen durch und geben Sie jeweils das Ergebnis aus: - Addition - Subtraktion - Multiplikation - Ganzzahlige Division (Ergebnis ohne Nachkommastellen) - Rest bei Division (Modulo) - Datentyp von `a` anzeigen

10.0.2 Hilfestellungen

Problemanalyse

- Welche Operationen sollen durchgeführt werden? – Grundrechenarten plus Modulo
- Welche Variablen werden benötigt? – Zwei Integer-Variablen `a` und `b`
- Was ist der Unterschied zwischen `/` und `//`? – `//` gibt nur den ganzzahligen Teil zurück
- Was macht `%`? – Gibt den Rest einer Division zurück

Lösungsschritte

1. Variable `a` mit Wert 10 erstellen
2. Variable `b` mit Wert 4 erstellen
3. Für jede Operation: Berechnung durchführen und mit `print()` ausgeben
4. Operatoren verwenden: `+`, `-`, `*`, `//`, `%`
5. Mit `type()` den Datentyp von `a` anzeigen

Pseudocode

```

a = 10
b = 4
AUSGABE "Addition:", a + b
AUSGABE "Subtraktion:", a - b
AUSGABE "Multiplikation:", a * b
AUSGABE "Ganzzahlige Division:", a // b
AUSGABE "Restwert (Modulo):", a % b
AUSGABE "Typ von a:", TYP VON a

```

10.0.3 Erwartete Ausgabe

```

Addition: 14
Subtraktion: 6
Multiplikation: 40
Ganzzahlige Division: 2
Restwert (Modulo): 2
Typ von a: <class 'int'>

```

10.1 Aufgabe 2 (Erweitert): Taschenrechner mit Eingabe

Schwierigkeit: Mittel

10.1.1 Aufgabenstellung

Erweitern Sie Ihr Programm so, dass der Benutzer zwei Zahlen eingeben kann. Das Programm soll dann alle Rechenoperationen mit diesen Zahlen durchführen und die Ergebnisse übersichtlich ausgeben.

10.1.2 Hilfestellungen

Problemanalyse

- Woher kommen die Zahlen? – Vom Benutzer über `input()`
- Warum braucht man `int()`? – `input()` gibt immer Strings zurück, zum Rechnen braucht man Integer
- Welche Berechnungen sollen durchgeführt werden? – Alle Grundrechenarten plus Modulo

Lösungsschritte

1. Trennlinie ausgeben
2. Erste Zahl vom Benutzer einlesen und mit `int()` umwandeln
3. Zweite Zahl vom Benutzer einlesen und mit `int()` umwandeln
4. Alle Rechenoperationen mit f-Strings ausgeben

Pseudocode

```

AUSGABE "--- Jetzt mit Eingabe ---"
a = INTEGER VON EINGABE "Erste Zahl: "
b = INTEGER VON EINGABE "Zweite Zahl: "
AUSGABE f"{a} + {b} = {a + b}"
AUSGABE f"{a} - {b} = {a - b}"
AUSGABE f"{a} * {b} = {a * b}"
AUSGABE f"{a} geteilt durch {b} (ganzzahlig) = {a // b}"
AUSGABE f"Rest von {a} / {b} = {a % b}"

```

10.1.3 Beispiel-Ausgabe (bei Eingabe 15 und 4)

```

--- Jetzt mit Eingabe ---
Erste Zahl: 15
Zweite Zahl: 4
15 + 4 = 19
15 - 4 = 11

```

```
15 * 4 = 60
15 geteilt durch 4 (ganzzahlig) = 3
Rest von 15 / 4 = 3
```

10.2 Datei ausführen und testen

```
python integer_uebung.py
```

10.3 Musterlösungen

Lösung zu Aufgabe 1

```
# Rechnen mit ganzen Zahlen
a = 10
b = 4

print("Addition:", a + b)
print("Subtraktion:", a - b)
print("Multiplikation:", a * b)
print("Ganzzahlige Division:", a // b) # Ergebnis ohne Rest
print("Restwert (Modulo):", a % b)    # Nur der Rest
print("Typ von a:", type(a))
```

Erklärung: - // gibt nur den ganzzahligen Teil zurück (2, nicht 2.5) - % gibt nur den Rest zurück (2, da $10 = 4 \cdot 2 + 2$) - type() zeigt, dass a ein Integer ist

Lösung zu Aufgabe 2 (Erweitert)

```
# Mit Benutzereingabe
print("\n--- Jetzt mit Eingabe ---")
a = int(input("Erste Zahl: "))
b = int(input("Zweite Zahl: "))

print(f"{a} + {b} = {a + b}")
print(f"{a} - {b} = {a - b}")
print(f"{a} * {b} = {a * b}")
print(f"{a} geteilt durch {b} (ganzzahlig) = {a // b}")
print(f"Rest von {a} / {b} = {a % b}")
```

Erklärung: - int(input()) konvertiert die String-Eingabe in einen Integer - f-Strings formatieren die Ausgabe übersichtlich

11 Übungsblock C – Float (ca. 15 Min)

11.1 Vorbereitung

1. Neue Datei erstellen: float_uebung.py
2. Datei im Editor öffnen

11.2 Aufgabe 1: Preisberechnung mit Mehrwertsteuer

12 Neue Funktion: round()

Die Funktion `round()` rundet eine Kommazahl auf eine bestimmte Anzahl von Nachkommastellen.

Syntax: `round(zahl, anzahl_stellen)`

Beispiele:

```
preis = 3.14159
print(round(preis, 2))    # Ausgabe: 3.14 (2 Nachkommastellen)
print(round(preis, 1))    # Ausgabe: 3.1 (1 Nachkommastelle)
print(round(preis))       # Ausgabe: 3 (keine Nachkommastellen)
```

Wichtig: Python verwendet "kaufmännisches Runden" (round half to even).

12.0.1 Aufgabenstellung

Ein Produkt kostet 4.99€ netto. Die Mehrwertsteuer beträgt 19% (= 0.19). Berechnen Sie: - Den Bruttopreis (Preis $\times$ (1 + Steuersatz)) - Zeigen Sie Nettopreis, Steuersatz und Bruttopreis an - Runden Sie den Bruttopreis auf 2 Nachkommastellen - Zeigen Sie den Datentyp der Preis-Variable an

12.0.2 Hilfestellungen

Problemanalyse

- Was soll berechnet werden? – Der Bruttopreis aus Nettopreis und Steuersatz
- Welche Formel gilt? – $\text{Brutto} = \text{Netto} \times (1 + \text{Steuersatz})$
- Warum runden? – Für eine saubere Geldbetragsdarstellung
- Wie schreibt man Kommazahlen in Python? – Mit Punkt, nicht mit Komma: 4.99

Lösungsschritte

1. Variable `preis` mit Wert 4.99 erstellen
2. Variable `steuer` mit Wert 0.19 erstellen
3. Berechnung durchführen: `preis * (1 + steuer)`
4. Ergebnis in `gesamtpreis` speichern
5. Alle Werte ausgeben (Preis, Steuer, Gesamtpreis, gerundet, Typ)

Pseudocode

```
preis = 4.99
steuer = 0.19
gesamtpreis = preis * (1 + steuer)
AUSGABE "Preis:", preis
AUSGABE "Steuersatz:", steuer
AUSGABE "Gesamtpreis:", gesamtpreis
AUSGABE "Gerundet (2 Stellen):", RUNDE gesamtpreis AUF 2 Stellen
AUSGABE "Typ von preis:", TYP VON preis
```

12.0.3 Erwartete Ausgabe

Preis: 4.99
Steuersatz: 0.19

```
Gesamtpreis: 5.9381
Gerundet (2 Stellen): 5.94
Typ von preis: <class 'float'>
```

12.1 Aufgabe 2 (Erweitert): Preisrechner mit Eingabe

Schwierigkeit: Mittel

12.1.1 Aufgabenstellung

Erweitern Sie das Programm, sodass der Benutzer einen Nettopreis und einen Steuersatz eingeben kann. Das Programm berechnet dann den Bruttopreis und gibt ihn gerundet aus.

12.1.2 Hilfestellungen

Problemanalyse

- Woher kommen die Werte? – Vom Benutzer über `input()`
- Warum `float()` statt `int()`? – Preise haben Nachkommastellen
- Wie gibt der Benutzer den Steuersatz ein? – Als Dezimalzahl (0.19 für 19%)

Lösungsschritte

1. Überschrift ausgeben
2. Nettopreis vom Benutzer einlesen und mit `float()` umwandeln
3. Steuersatz vom Benutzer einlesen und mit `float()` umwandeln
4. Bruttopreis berechnen
5. Nettopreis und Bruttopreis mit f-Strings und Euro-Symbol ausgeben

Pseudocode

```
AUSGABE "--- Preisberechnung mit Eingabe ---"
preis = FLOAT VON EINGABE "Nettopreis eingeben: "
steuer = FLOAT VON EINGABE "Steuersatz eingeben..."
gesamtpreis = preis * (1 + steuer)
AUSGABE f"Nettopreis: {preis}€"
AUSGABE f"Bruttopreis: {RUNDE gesamtpreis AUF 2}€"
```

12.1.3 Beispiel-Ausgabe (bei Eingabe 10.50 und 0.19)

```
--- Preisberechnung mit Eingabe ---
Nettopreis eingeben: 10.50
Steuersatz eingeben (z.B. 0.19 für 19%): 0.19
Nettopreis: 10.5€
Bruttopreis: 12.5€
```

12.2 Datei ausführen und testen

python float_uebung.py

12.3 Musterlösungen

Lösung zu Aufgabe 1

```
# Rechnen mit Kommazahlen
preis = 4.99
steuer = 0.19
gesamtpreis = preis * (1 + steuer)
```

```
print("Preis:", preis)
print("Steuersatz:", steuer)
print("Gesamtpreis:", gesamtpreis)
print("Gerundet (2 Stellen):", round(gesamtpreis, 2))
print("Typ von preis:", type(preis))
```

Erklärung: - $(1 + \text{steuer})$ entspricht 119% = 1.19 - round() rundet auf kaufmännische Art - type() zeigt <class 'float'> für Kommazahlen

Lösung zu Aufgabe 2 (Erweitert)

```
# Mit Benutzereingabe
print("\n--- Preisberechnung mit Eingabe ---")
preis = float(input("Nettopreis eingeben: "))
steuer = float(input("Steuersatz eingeben (z.B. 0.19 für 19%): "))
gesamtpreis = preis * (1 + steuer)

print(f"Nettopreis: {preis}€")
print(f"Bruttopreis: {round(gesamtpreis, 2)}€")
```

Erklärung: - float(input()) konvertiert die Eingabe in eine Kommazahl - Der Bruttopreis wird direkt gerundet ausgegeben

13 Übungsblock D – Boolean (ca. 10 Min)

13.1 Vorbereitung

1. Neue Datei erstellen: `boolean_uebung.py`
2. Datei im Editor öffnen

13.2 Aufgabe: Logische Verknüpfungen verstehen

14 Logische Operatoren: `and`, `or`, `not`

Mit logischen Operatoren können Boolean-Werte kombiniert werden.

and (UND): Beide Bedingungen müssen `True` sein

```
print(True and True)    # Ausgabe: True
print(True and False)   # Ausgabe: False
```

or (ODER): Mindestens eine Bedingung muss `True` sein

```
print(True or False)    # Ausgabe: True
print(False or False)   # Ausgabe: False
```

not (NICHT): Kehrt den Wahrheitswert um

```
print(not True)         # Ausgabe: False
print(not False)        # Ausgabe: True
```

15 Vergleichsoperatoren

Vergleiche liefern Boolean-Werte (`True` oder `False`) zurück:

Tabelle 2: Vergleichsoperatoren in Python {#tbl:vergleiche}

Operator	Bedeutung	Beispiel
<code>==</code>	Gleich	<code>5 == 5</code> – <code>True</code>
<code>!=</code>	Ungleich	<code>5 != 3</code> – <code>True</code>
<code><</code>	Kleiner als	<code>3 < 5</code> – <code>True</code>
<code>></code>	Größer als	<code>5 > 3</code> – <code>True</code>
<code><=</code>	Kleiner oder gleich	<code>3 <= 3</code> – <code>True</code>
<code>>=</code>	Größer oder gleich	<code>5 >= 5</code> – <code>True</code>

Wichtig: `==` prüft auf Gleichheit, `=` ist Zuweisung!

15.0.1 Aufgabenstellung

Erstellen Sie zwei Boolean-Variablen: - `ist_student` mit dem Wert `True` - `hat_ausweis` mit dem Wert `False`

Führen Sie dann folgende logische Operationen durch: 1. Zeigen Sie beide Variablen einzeln an 2. UND-Verknüpfung (`and`) 3. ODER-Verknüpfung (`or`) 4. Negation (`not`) 5. Zeigen Sie den Datentyp an

Zusätzlich: Erstellen Sie eine Variable `alter` mit dem Wert 20 und führen Sie Vergleichsoperationen durch.

15.0.2 Hilfestellungen

Problemanalyse

- Was sind Boolean-Werte? – Wahrheitswerte: nur **True** oder **False**
- Was macht **and**? – Beide Bedingungen müssen wahr sein
- Was macht **or**? – Mindestens eine Bedingung muss wahr sein
- Was macht **not**? – Kehrt den Wahrheitswert um (True – False)
- Wie entstehen Booleans? – Durch Vergleiche wie **>=**, **<**, **==**

Lösungsschritte

1. Variable **ist_student** mit **True** erstellen
2. Variable **hat_ausweis** mit **False** erstellen
3. Beide Variablen einzeln ausgeben
4. Logische Operationen **and**, **or**, **not** anwenden und ausgeben
5. Datentyp mit **type()** anzeigen
6. Variable **alter** mit 20 erstellen
7. Vergleichsoperationen durchführen und ausgeben

Pseudocode

```
ist_student = True
hat_ausweis = False
AUSGABE "Ist Student:", ist_student
AUSGABE "Hat Ausweis:", hat_ausweis
AUSGABE "Student UND Ausweis:", ist_student AND hat_ausweis
AUSGABE "Student ODER Ausweis:", ist_student OR hat_ausweis
AUSGABE "NICHT Ausweis:", NOT hat_ausweis
AUSGABE "Typ von ist_student:", TYP VON ist_student

alter = 20
AUSGABE "Vergleiche:"
AUSGABE "alter >= 18:", alter >= 18
AUSGABE "alter < 18:", alter < 18
AUSGABE "alter == 20:", alter == 20
```

15.0.3 Erwartete Ausgabe

```
Ist Student: True
Hat Ausweis: False
Student UND Ausweis: False
Student ODER Ausweis: True
NICHT Ausweis: True
Typ von ist_student: <class 'bool'>
```

Vergleiche:

```
alter >= 18: True
alter < 18: False
alter == 20: True
```

15.1 Datei ausführen und testen

```
python boolean_uebung.py
```

15.2 Musterlösungen

Lösung

```
# Wahrheitswerte und logische Verknüpfungen
ist_student = True
hat_ausweis = False

print("Ist Student:", ist_student)
print("Hat Ausweis:", hat_ausweis)
print("Student UND Ausweis:", ist_student and hat_ausweis)
print("Student ODER Ausweis:", ist_student or hat_ausweis)
print("NICHT Ausweis:", not hat_ausweis)
print("Typ von ist_student:", type(ist_student))

# Vergleichsoperationen ergeben auch Boolean-Werte
alter = 20
print("\nVergleiche:")
print("alter >= 18:", alter >= 18)
print("alter < 18:", alter < 18)
print("alter == 20:", alter == 20)
```

Erklärung: - and: Nur True wenn beide True sind - or: True wenn mindestens einer True ist - not: Dreht False zu True um - Vergleiche geben immer Boolean-Werte zurück

16 Übungsblock E – Typprüfung & Umwandlung (ca. 15 Min)

16.1 Vorbereitung

1. Neue Datei erstellen: `typumwandlung_uebung.py`
2. Datei im Editor öffnen

16.2 Aufgabe: Datentypen umwandeln

17 Typumwandlung: `str()`, `int()`, `float()`, `bool()`

Mit diesen Funktionen können Werte zwischen verschiedenen Datentypen umgewandelt werden.

`str()` - In String umwandeln:

```
zahl = 42
text = str(zahl)
print(text)           # Ausgabe: "42"
print(type(text))     # Ausgabe: <class 'str'>
```

`int()` - In Integer umwandeln:

```
text = "100"
zahl = int(text)
print(zahl)           # Ausgabe: 100

# Bei Float: Nachkommastellen werden ABGESCHNITTEN!
print(int(3.99))      # Ausgabe: 3 (nicht 4!)
```

`float()` - In Float umwandeln:

```
text = "3.14"
zahl = float(text)
print(zahl)           # Ausgabe: 3.14
```

`bool()` - In Boolean umwandeln:

```
print(bool(1))        # Ausgabe: True
print(bool(0))        # Ausgabe: False
print(bool(""))       # Ausgabe: False (leerer String)
print(bool("Hallo"))  # Ausgabe: True
```

Wichtig: - `int("3.5")` funktioniert NICHT! – Erst `float()`, dann `int()` - `True` wird zu 1, `False` wird zu 0

17.0.1 Aufgabenstellung

Erstellen Sie verschiedene Variablen und wandeln Sie sie zwischen Datentypen um:

1. Integer zu String umwandeln (42)
2. Float zu Integer umwandeln (3.99) - Was passiert mit den Nachkommastellen?
3. Boolean zu Integer umwandeln (True und False)
4. String zu Integer umwandeln ("100")

17.0.2 Hilfestellungen

Problemanalyse

- Was bedeutet Typumwandlung? – Den Datentyp eines Wertes ändern
- Welche Funktionen gibt es? – `str()`, `int()`, `float()`, `bool()`
- Was passiert bei `int()` mit Kommazahlen? – Nachkommastellen werden abgeschnitten (nicht gerundet!)
- Was wird aus `True/False` bei `int()`? – 1 bzw. 0

Lösungsschritte

1. Variable `x` mit 42 erstellen
2. `x` und seinen Typ ausgeben
3. `x` mit `str()` in String umwandeln und ausgeben
4. Variable `y` mit 3.99 erstellen
5. `y` mit `int()` in Integer umwandeln und zeigen, was passiert
6. `True` und `False` mit `int()` umwandeln
7. String "100" mit `int()` umwandeln

Pseudocode

```

x = 42
AUSGABE "x =", x
AUSGABE "Typ von x:", TYP VON x
x_str = STRING VON x
AUSGABE "x als String:", x_str, "| Typ:", TYP VON x_str

y = 3.99
AUSGABE "y =", y
AUSGABE "y als Integer:", INTEGER VON y

AUSGABE "Boolean zu Integer:"
AUSGABE "int(True) =", INTEGER VON True
AUSGABE "int(False) =", INTEGER VON False

zahl_text = "100"
zahl = INTEGER VON zahl_text
AUSGABE "String '100' zu Integer:", zahl, "| Typ:", TYP VON zahl

```

17.0.3 Erwartete Ausgabe

```

x = 42
Typ von x: <class 'int'>
x als String: 42 | Typ: <class 'str'>

y = 3.99
y als Integer: 3

Boolean zu Integer:
int(True) = 1
int(False) = 0

String '100' zu Integer: 100 | Typ: <class 'int'>

```

17.1 Datei ausführen und testen

```
python typumwandlung_uebung.py
```

17.2 Musterlösungen

Lösung

```

# Typen prüfen und umwandeln
x = 42
print("x =", x)
print("Typ von x:", type(x))

```

```
# Integer zu String
x_str = str(x)
print("x als String:", x_str, "| Typ:", type(x_str))

# Float zu Integer (Nachkommastellen werden abgeschnitten!)
y = 3.99
print("\ny =", y)
print("y als Integer:", int(y))

# Boolean zu Integer
print("\nBoolean zu Integer:")
print("int(True) =", int(True))
print("int(False) =", int(False))

# String zu Integer (nur wenn möglich!)
zahl_text = "100"
zahl = int(zahl_text)
print("\nString '100' zu Integer:", zahl, "| Typ:", type(zahl))
```

Erklärung: - `int()` schneidet Nachkommastellen ab (3.99 – 3) - `True` wird zu 1, `False` zu 0 konvertiert
- String-zu-Integer funktioniert nur bei gültigen Zahlen

18 Übungsblock F – Gemischte Aufgaben (ca. 25 Min)

18.1 Aufgabe 1: Temperaturumrechner

Schwierigkeit: Einfach

18.1.1 Vorbereitung

1. Neue Datei erstellen: `temperatur_umrechner.py`

18.1.2 Aufgabenstellung

Erstellen Sie ein Programm, das Celsius in Fahrenheit umrechnet: - Benutzer gibt eine Temperatur in Celsius ein - Programm berechnet den Wert in Fahrenheit - Formel: $\text{Fahrenheit} = \text{Celsius} \times 9/5 + 32$ - Ergebnis wird gerundet auf 2 Nachkommastellen ausgegeben

18.1.3 Hilfestellungen

Problemanalyse

- Was soll umgerechnet werden? – Celsius nach Fahrenheit
- Welche Formel gilt? – $\text{Fahrenheit} = \text{Celsius} \times 9/5 + 32$
- Woher kommt der Celsius-Wert? – Vom Benutzer über `input()`
- Wie wird die Ausgabe formatiert? – Mit f-String und den Grad-Symbolen °C und °F

Lösungsschritte

1. Überschrift “=== Temperaturumrechner ===” ausgeben
2. Celsius-Wert vom Benutzer einlesen mit `float(input(...))`
3. Formel anwenden: `celsius * 9/5 + 32`
4. Ergebnis auf 2 Nachkommastellen runden
5. Mit f-String beide Werte ausgeben

Pseudocode

```
AUSGABE "=== Temperaturumrechner ==="
celsius = FLOAT VON EINGABE "Temperatur in °C: "
fahrenheit = celsius * 9/5 + 32
AUSGABE f"{celsius}°C = {RUNDE fahrenheit AUF 2}°F"
```

18.1.4 Beispiel-Ausgabe (bei 0°C und 100°C)

```
=== Temperaturumrechner ===
Temperatur in °C: 0
0.0°C = 32.0°F
```

```
=== Temperaturumrechner ===
Temperatur in °C: 100
100.0°C = 212.0°F
```

18.1.5 Testwerte zum Überprüfen

- 0°C sollte 32°F ergeben
- 100°C sollte 212°F ergeben
- 20°C sollte 68°F ergeben

18.1.6 Datei ausführen und testen

```
python temperatur_umrechner.py
```

18.2 Aufgabe 2: Notenprüfung

Schwierigkeit: Mittel

18.2.1 Vorbereitung

1. Neue Datei erstellen: `noten_pruefung.py`

18.2.2 Aufgabenstellung

Erstellen Sie ein Programm zur Prüfung, ob eine Note zum Bestehen ausreicht: - Benutzer gibt eine Note zwischen 1 und 6 ein - Programm prüft, ob die Note ≤ 4 ist (bestanden) - Gibt den Boolean-Wert aus (True/False)

18.2.3 Hilfestellungen

Problemanalyse

- Was soll geprüft werden? – Ob die Note zum Bestehen ausreicht
- Welche Bedingung gilt für “bestanden”? – Note kleiner oder gleich 4
- Was ist das Ergebnis? – Ein Boolean-Wert (True = bestanden, False = nicht bestanden)
- Woher kommt die Note? – Vom Benutzer über `input()`

Lösungsschritte

1. Überschrift “=== Notenprüfung ===” ausgeben
2. Note vom Benutzer einlesen mit `int(input(...))`
3. Vergleich durchführen: `note <= 4`
4. Ergebnis in Variable `bestanden` speichern
5. Note und Ergebnis ausgeben

Pseudocode

```
AUSGABE "=== Notenprüfung ==="
note = INTEGER VON EINGABE "Note eingeben (1-6): "
bestanden = note <= 4
AUSGABE f"Note: {note}"
AUSGABE f"Bestanden: {bestanden}"
```

18.2.4 Beispiel-Ausgabe (bei Note 3 und Note 5)

```
=== Notenprüfung ===
Note eingeben (1-6): 3
Note: 3
Bestanden: True
```

```
=== Notenprüfung ===
Note eingeben (1-6): 5
Note: 5
Bestanden: False
```

18.2.5 Datei ausführen und testen

`python noten_pruefung.py`

18.3 Musterlösungen

Lösung zu Aufgabe 1: Temperaturumrechner

```
# Temperaturumrechner Celsius -- Fahrenheit
print("=== Temperaturumrechner ===")
celsius = float(input("Temperatur in °C: "))
```

```
fahrenheit = celsius * 9/5 + 32
print(f"{celsius}°C = {round(fahrenheit, 2)}°F")
```

Erklärung: - Formel wird direkt umgesetzt - Punkt-vor-Strich wird automatisch beachtet - `round()` sorgt für saubere Ausgabe

Lösung zu Aufgabe 2: Notenprüfung

```
# Notenprüfung
print("=== Notenprüfung ===")
note = int(input("Note eingeben (1-6): "))
bestanden = note <= 4
print(f"Note: {note}")
print(f"Bestanden: {bestanden}")

# Hinweis: Die Textausgabe könnte mit if-else erfolgen,
# das lernen Sie aber erst in Notebook 09.
# Für jetzt reicht die Ausgabe des Boolean-Wertes.
```

Erklärung: - `note <= 4` ergibt direkt einen Boolean-Wert - Dieser wird in `bestanden` gespeichert - Die Textausgabe mit `if-else` kommt erst in Notebook 09

19 Typische Anfängerfehler – Troubleshooting

19.1 Python-Fehler

Tabelle 3: Häufige Python-Fehler und ihre Lösungen {#tbl:fehler}

Fehler	Ursache	Lösung
<code>NameError: name 'x' is not defined</code>	Variable wurde nicht initialisiert oder Tippfehler	Variable vor Verwendung mit Wert belegen; Schreibweise prüfen
<code>TypeError: can only concatenate str (not "int") to str</code>	Versuch, String mit Integer zu kombinieren	Typumwandlung verwenden: <code>str(zahl)</code>
<code>ValueError: invalid literal for int()</code>	Versuch, ungültigen String in Integer umzuwandeln	Erst zu Float: <code>int(float("3.5"))</code>
<code>SyntaxError: invalid syntax</code>	Fehlende Klammern, Anführungszeichen oder Doppelpunkt	Syntaxprüfung; öffnende und schließende Zeichen zählen
<code>IndentationError</code>	Falsche Einrückung	Konsistente Einrückung (4 Leerzeichen)
<code>ZeroDivisionError</code>	Division durch Null	Vor Division prüfen, ob Nenner <code>!= 0</code>

19.2 VSCode-spezifische Probleme

Tabelle 4: VSCode-Probleme und Lösungen {#tbl:vscode}

Problem	Ursache	Lösung
Datei wird nicht ausgeführt	Python nicht installiert oder nicht im PATH	Python installieren; ggf. <code>python3</code> verwenden
Terminal zeigt alten Code	Terminal-Cache	Terminal leeren mit <code>cls</code> oder <code>clear</code>
Änderungen nicht übernommen	Datei nicht gespeichert	<code>Strg + S</code> zum Speichern
Falsches Terminal-Verzeichnis	Terminal im falschen Ordner	<code>cd python_grundlagen</code>
Code wird nicht farbig	Datei nicht als <code>.py</code> gespeichert	Mit <code>.py</code> -Endung speichern

19.3 Eingabefehler bei `input()`

Tabelle 5: Eingabefehler und Lösungen {#tbl:eingabe}

Problem	Ursache	Lösung
<code>ValueError</code> bei <code>int(input())</code>	Buchstaben statt Zahlen eingegeben	Nur Zahlen eingeben
Kommazahl nicht erkannt	Komma statt Punkt	Punkt verwenden: <code>3.14</code> statt <code>3,14</code>
<code>EOFError</code>	Programm wartet auf Eingabe	In Terminal eingeben und Enter drücken

19.4 Tipps zur Fehlervermeidung

Goldene Regeln: 1. - [x] **Immer speichern** (`Strg + S`) bevor Sie ausführen 2. - [x] **Fehlermeldungen lesen** – sie zeigen oft die Zeile an 3. - [x] **Schritt für Schritt** – testen Sie jede Zeile einzeln 4. - [x] **Variablennamen prüfen** – `Name != name` 5. - [x] **Anführungszeichen schließen** – jedes " braucht ein schließendes " 6. - [x] **Klammern zählen** – jede (braucht eine) 7. - [x] **Bei `input()` den richtigen Typ verwenden:** - Für Zahlen: `int(input())` oder `float(input())` - Für Text: einfach `input()`

20 Abschluss / Reflexion (ca. 10 Min)

20.1 Alle Skripte testen

Checkliste zum Durchgehen: - [] Jede erstellte Datei noch einmal ausführen - [] Verschiedene Eingabewerte ausprobieren - [] Terminal bei Bedarf löschen (`cls` oder `clear`)

20.2 Reflexionsfragen

Nehmen Sie sich einen Moment Zeit und überlegen Sie:

1. **Welche Datentypen gibt es in Python?**
 - String, Integer, Float, Boolean
2. **Wie prüft man den Typ einer Variable?**
 - Mit der Funktion `type()`
3. **Wie wandelt man Werte zwischen Typen um?**
 - Mit `str()`, `int()`, `float()`, `bool()`
4. **Wann muss man `int()` oder `float()` bei `input()` verwenden?**
 - Wenn man mit den eingegebenen Werten rechnen möchte
5. **Welche typischen Fehler können auftreten und wie behebt man sie?**
 - `NameError` – Variable definieren
 - `TypeError` – Typumwandlung
 - `ValueError` – Gültige Werte verwenden

20.3 Dateien organisieren

Überprüfen Sie, dass alle Übungsdateien im Ordner `python_grundlagen` liegen:

Fortschritt: 90% abgeschlossen

Erstellte Dateien:

- ☒ `hello_world.py`
- ☒ `strings_uebung.py`
- ☒ `integer_uebung.py`
- ☒ `float_uebung.py`
- ☒ `boolean_uebung.py`
- ☒ `typumwandlung_uebung.py`
- ☒ `temperatur_umrechner.py`
- ☒ `noten_pruefung.py`

20.4 Was haben Sie gelernt?

In dieser Übungseinheit haben Sie: - Die vier grundlegenden Datentypen kennengelernt - Mit Variablen gearbeitet - Zwischen Datentypen umgewandelt - Erste kleine Programme geschrieben - Benutzereingaben verarbeitet

Herzlichen Glückwunsch! Sie haben die Grundlagen der Datentypen in Python gemeistert!

20.5 Nächste Schritte

Im nächsten Notebook lernen Sie: - Listen und andere komplexe Datentypen - Wie man mehrere Werte speichert - Wie man mit Sammlungen von Daten arbeitet

Ende der Übungseinheit